

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 9: Dynamic Programming II: Sequence Alignment and Shortest Paths

Foreign Trade University

Contents

1	Introduction: Two-Dimensional Dynamic Programming	1
1.1	The recurring recipe	1
2	Sequence Alignment and Edit Distance	2
2.1	Problem statement	2
2.2	The structural insight	2
2.3	The recurrence	2
2.4	Running time and the table	3
2.5	Reconstructing an optimal alignment	3
2.6	Hirschberg’s space-saving idea	3
3	Longest Common Subsequence	4
3.1	Problem statement	4
3.2	The recurrence	4
3.3	The LCS–edit-distance relationship	4
4	Shortest Paths with Negative Edges: Bellman–Ford	5
4.1	Why Dijkstra is not enough	5
4.2	The DP formulation	5
4.3	The algorithm	5
4.4	Negative-cycle detection	6
5	All-Pairs Shortest Paths: Floyd–Warshall	6
5.1	Problem and recurrence	6
6	Shortest and Longest Paths in a DAG	7
6.1	Topological order makes DP trivial	7
7	Two More Classic 2-D Dynamic Programs	8
7.1	Matrix chain multiplication	8
7.2	Grid paths	8
8	Summary and Looking Ahead	8

1 Introduction: Two-Dimensional Dynamic Programming

Last week (Dynamic Programming I) we introduced the dynamic-programming paradigm through problems whose subproblems were indexed by a *single* parameter – weighted interval scheduling (“the best solution using the first i requests”) and the knapsack problem (“the best value achievable with the first i items and capacity w ”). This week (following Kleinberg & Tardos, Ch. 6) we push the same idea further in two directions.

First, we study **sequence alignment** (a.k.a. *edit distance*), where the subproblems are indexed by *two* parameters – a prefix of one string and a prefix of another – giving a two-dimensional table. This single recurrence underlies the diff utility, spell checkers, and biological sequence comparison (DNA/protein alignment), and its close cousin the **longest common subsequence (LCS)** problem.

Second, we revisit **shortest paths**, but now in a setting Dijkstra’s algorithm *cannot* handle: graphs with **negative edge weights**. We will see that shortest paths have a clean DP formulation – the **Bellman–Ford** algorithm – that not only tolerates negative edges but can also *detect negative cycles*. We then generalise to all pairs of vertices with the elegant **Floyd–Warshall** algorithm, and finish with the special (and especially fast) case of shortest/longest paths in a **directed acyclic graph (DAG)**.

1.1 The recurring recipe

Every algorithm this week follows the same four-step DP recipe:

- (1) **Define subproblems.** Identify a small set of subproblems, each indexed by one or more parameters, whose optimal solutions can be combined into a solution of the whole.
- (2) **Write a recurrence.** Express the optimal value of a subproblem in terms of the optimal values of “smaller” subproblems, by reasoning about the *last decision* an optimal solution must make.
- (3) **Order the computation.** Fill in a table so that every subproblem is solved before it is needed (smallest first, or in topological order).
- (4) **Reconstruct.** Trace back through the table (or store “parent” pointers) to recover an optimal solution itself, not just its value.

The hard step is almost always (2): finding the right way to reason about the last decision. Once the recurrence is correct, the implementation is mechanical and the running time is just (number of subproblems) $\times$ (work per subproblem).

2 Sequence Alignment and Edit Distance

2.1 Problem statement

Definition 2.1 (Edit Distance / Sequence Alignment). *Given two strings $X = x_1x_2\cdots x_m$ and $Y = y_1y_2\cdots y_n$, an alignment is a way of writing the two strings one above the other, inserting gap symbols (“–”) into either string, so that the two padded strings have the same length and no column consists of two gaps. Each column then either matches two equal characters, mismatches two unequal characters, or pairs a character with a gap (an insertion or deletion). Given a gap cost $\delta \geq 0$ and a mismatch cost $\alpha_{pq} \geq 0$ for substituting character p by q (with $\alpha_{pp} = 0$), the cost*

of an alignment is the sum of its column costs, and the goal is to find an alignment of **minimum total cost**.

The **Levenshtein edit distance** is the special case $\delta = 1$ and $\alpha_{pq} = 1$ for all $p \neq q$: it counts the minimum number of single-character *insertions*, *deletions*, and *substitutions* needed to turn X into Y . Problem 1 asks you to implement exactly this; Problem 2 generalises to arbitrary δ and α .

2.2 The structural insight

Consider an optimal alignment of X and Y , and look at its *last* column. There are exactly three possibilities for what happens to x_m and y_n :

- x_m and y_n are aligned together (a match if $x_m = y_n$, else a mismatch);
- x_m is aligned with a gap (x_m is deleted);
- y_n is aligned with a gap (y_n is inserted).

Crucially, no other column may involve *both* x_m and y_n : alignments preserve order, so once we have decided what happens to the last characters, the rest of the alignment is an optimal alignment of the two remaining prefixes. This “what happens to the last character” reasoning is the heart of the recurrence.

2.3 The recurrence

Let $\text{OPT}(i, j)$ denote the minimum cost of aligning the prefix $x_1 \cdots x_i$ with the prefix $y_1 \cdots y_j$. Then:

$$\text{OPT}(i, j) = \min \begin{cases} \alpha_{x_i y_j} + \text{OPT}(i-1, j-1) & (\text{align } x_i \text{ with } y_j) \\ \delta + \text{OPT}(i-1, j) & (\text{delete } x_i, \text{ gap in } Y) \\ \delta + \text{OPT}(i, j-1) & (\text{insert } y_j, \text{ gap in } X) \end{cases}$$

with base cases $\text{OPT}(i, 0) = i\delta$ (delete all of X ’s first i characters) and $\text{OPT}(0, j) = j\delta$ (insert all of Y ’s first j characters). The answer is $\text{OPT}(m, n)$.

Theorem 2.2. *The recurrence above correctly computes the minimum-cost alignment of X and Y .*

Proof. We argue that $\text{OPT}(i, j)$ equals the true optimal alignment cost of $x_1 \cdots x_i$ and $y_1 \cdots y_j$, by induction on $i + j$. The base cases ($i = 0$ or $j = 0$) are forced: aligning a non-empty prefix with the empty string requires exactly one gap per character, at total cost $i\delta$ (resp. $j\delta$). For the inductive step with $i, j \geq 1$, consider any optimal alignment A of the two prefixes and examine its last column. By the structural insight, that column is one of the three cases above. If it aligns x_i with y_j , then deleting this column leaves an alignment of $x_1 \cdots x_{i-1}$ and $y_1 \cdots y_{j-1}$, which must itself be optimal (else we could improve A); by induction its cost is $\text{OPT}(i-1, j-1)$, so A costs $\alpha_{x_i y_j} + \text{OPT}(i-1, j-1)$. The other two cases are symmetric and give the other two terms. Hence the cost of A is at least the minimum of the three expressions, so $\text{OPT}(i, j) \leq \text{cost}(A)$. Conversely, each of the three expressions corresponds to a *valid* alignment (take an optimal alignment of the relevant smaller prefixes and append the appropriate last column), so $\text{OPT}(i, j)$ is achievable. Therefore $\text{OPT}(i, j)$ equals the optimum. $\square$

2.4 Running time and the table

There are $(m + 1)(n + 1)$ subproblems, each solved in $O(1)$ time from three already-computed neighbours (the cell directly above, to the left, and diagonally up-left). Filling the table row by row (or column by column) therefore takes $\mathbf{O(mn)}$ time and $O(mn)$ space. This is a quadratic-time algorithm for a problem whose naive solution (enumerating all alignments) is exponential.

2.5 Reconstructing an optimal alignment

The value $\text{OPT}(m, n)$ is only the *cost*. To recover the alignment itself (Problem 3), trace back from cell (m, n) to $(0, 0)$: at each cell (i, j) , check which of the three terms attained the minimum, emit the corresponding column (a match/mismatch, a deletion, or an insertion), and move to the predecessor cell. Reversing the emitted columns gives the two aligned strings (with gap symbols). The traceback visits at most $m + n$ cells, so reconstruction is $O(m + n)$ given the filled table.

Algorithm 1 Sequence Alignment (fill table, then trace back)

```

1:  $\text{OPT}[i][0] \leftarrow i \delta$  for all  $i$ ;  $\text{OPT}[0][j] \leftarrow j \delta$  for all  $j$ 
2: for  $i = 1$  to  $m$  do
3:   for  $j = 1$  to  $n$  do
4:      $\text{OPT}[i][j] \leftarrow \min(\alpha_{x_i y_j} + \text{OPT}[i-1][j-1], \delta + \text{OPT}[i-1][j], \delta + \text{OPT}[i][j-1])$ 
5:   end for
6: end for
7: Trace back from  $(m, n)$  to  $(0, 0)$ , emitting one column per step
8: return  $\text{OPT}[m][n]$  and the reconstructed alignment

```

2.6 Hirschberg's space-saving idea

For very long strings (e.g. genomic sequences with m, n in the millions) the $O(mn)$ *space* is the binding constraint, not the time. A beautiful observation rescues us if we only need the optimal *cost*: each row of the table depends only on the previous row, so we can keep just two rows (or even one, updated in place) and compute $\text{OPT}(m, n)$ in $O(\min(m, n))$ space. But then we lose the information needed to trace back the alignment.

Hirschberg's algorithm recovers the full alignment in $O(mn)$ time and only $O(m + n)$ space by combining this space-saving trick with divide-and-conquer. The key fact: an optimal alignment of X and Y must pass through some cell $(m/2, q)$ in the middle column. The optimal q can be found by computing, in linear space, the cost of aligning the first half of X with each prefix of Y (a forward pass) and the cost of aligning the second half of X with each suffix of Y (a backward pass), then choosing the q minimising their sum. The problem then splits into two independent halves ($x_1 \cdots x_{m/2}$ vs. $y_1 \cdots y_q$, and the rest), each solved recursively. The recurrence for the total work is $T(m, n) = O(mn) + T(m/2, q) + T(m/2, n - q)$, which solves to $O(mn)$ – the same asymptotic time, but now in linear space. We mention this only as a pointer; the details are in Kleinberg & Tardos, Section 6.7.

3 Longest Common Subsequence

3.1 Problem statement

Definition 3.1 (Longest Common Subsequence). A subsequence of a string is obtained by deleting zero or more characters without reordering the rest. Given $X = x_1 \cdots x_m$ and $Y = y_1 \cdots y_n$, the longest common subsequence (LCS) problem asks for a longest string that is a subsequence of both.

For example, the LCS of AGGTAB and GXTXAYB is GTAB (length 4).

3.2 The recurrence

Let $L(i, j)$ be the length of the LCS of $x_1 \cdots x_i$ and $y_1 \cdots y_j$. Reasoning again about the last characters:

$$L(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ 1 + L(i - 1, j - 1) & \text{if } x_i = y_j, \\ \max(L(i - 1, j), L(i, j - 1)) & \text{if } x_i \neq y_j. \end{cases}$$

If $x_i = y_j$ we may safely match them (one can prove some optimal LCS does so); otherwise at least one of x_i, y_j is unused, giving the two max terms. The table is $O(mn)$ to fill, and an actual LCS string is recovered by the same kind of traceback as for alignment (Problem 4). LCS is itself a special case of the alignment framework: it is exactly the *maximum-score* alignment with match reward +1, mismatch 0, and gap 0.

3.3 The LCS–edit-distance relationship

There is a clean identity when the only edit operations are insertion and deletion (no substitution – equivalently, substitution costs 2, more than a delete-plus-insert):

Theorem 3.2. Let $d(X, Y)$ be the minimum number of single-character insertions and deletions to turn X into Y . Then

$$d(X, Y) = m + n - 2 \cdot \text{LCS}(X, Y),$$

where $\text{LCS}(X, Y)$ is the length of the longest common subsequence.

Proof sketch. Any common subsequence S of length ℓ gives an edit script: delete the $m - \ell$ characters of X not in S , then insert the $n - \ell$ characters of Y not in S , for $(m - \ell) + (n - \ell) = m + n - 2\ell$ operations. Maximising ℓ minimises the count, so $d(X, Y) \leq m + n - 2 \text{LCS}$. Conversely, any insert/delete script that converts X to Y leaves a set of untouched characters forming a common subsequence of length ℓ' , and uses at least $m + n - 2\ell'$ operations, so $d(X, Y) \geq m + n - 2 \text{LCS}$. $\square$

Problem 5 asks you to verify this identity empirically by comparing an insert/delete-only edit distance against $m + n - 2 \text{LCS}$ on random strings.

4 Shortest Paths with Negative Edges: Bellman–Ford

4.1 Why Dijkstra is not enough

Dijkstra’s algorithm (Week 5) assumes all edge weights are non-negative; it commits to a vertex’s distance as soon as that vertex is extracted from the priority queue, which is only valid when no later (negative) edge could improve it. With negative edges this assumption fails, and Dijkstra

can return wrong answers. Negative edges arise naturally – e.g. in **arbitrage detection** (currency exchange rates become edge weights after taking logarithms, and a profitable cycle is a negative cycle) and in problems reduced to shortest paths.

A subtlety: if a graph contains a **negative cycle** reachable on the way from s to t , the notion of “shortest path” is ill-defined – one can loop around the cycle arbitrarily many times to drive the cost to $-\infty$. So the right problem is: *find shortest paths assuming no negative cycle, and otherwise report that one exists.*

4.2 The DP formulation

The key idea is to bound the *number of edges* a shortest path may use. In a graph with no negative cycle, any shortest s – v path is *simple* (visits no vertex twice), hence uses at most $n - 1$ edges. So define

$\text{OPT}(k, v)$ = the minimum cost of an s – v path using at most k edges.

Reasoning about the *last edge* of such a path:

$$\text{OPT}(k, v) = \min \left(\text{OPT}(k - 1, v), \min_{(u,v) \in E} (\text{OPT}(k - 1, u) + w(u, v)) \right),$$

with $\text{OPT}(0, s) = 0$ and $\text{OPT}(0, v) = \infty$ for $v \neq s$. Either the shortest path with $\leq k$ edges actually uses $\leq k - 1$ edges (first term), or its last edge is some (u, v) and the prefix is an optimal $\leq k - 1$ edge path to u (second term). The true shortest-path distances are $\text{OPT}(n - 1, v)$.

4.3 The algorithm

In implementation we do not keep a separate row per k ; we keep a single distance array $\text{dist}[\cdot]$ and **relax** every edge, repeating the whole pass $n - 1$ times. “Relaxing” edge (u, v) means: if $\text{dist}[u] + w(u, v) < \text{dist}[v]$, update $\text{dist}[v]$ (and record u as v ’s parent).

Algorithm 2 Bellman–Ford (single-source shortest paths)

```

1:  $\text{dist}[s] \leftarrow 0$ ;  $\text{dist}[v] \leftarrow \infty$  for all  $v \neq s$ 
2: for  $k = 1$  to  $n - 1$  do
3:   for each edge  $(u, v, w) \in E$  do
4:     if  $\text{dist}[u] + w < \text{dist}[v]$  then
5:        $\text{dist}[v] \leftarrow \text{dist}[u] + w$ ;  $\text{parent}[v] \leftarrow u$ 
6:     end if
7:   end for
8: end for
9:                                      $\triangleright$  Negative-cycle check: one more pass
10: for each edge  $(u, v, w) \in E$  do
11:   if  $\text{dist}[u] + w < \text{dist}[v]$  then
12:     return “negative cycle detected”
13:   end if
14: end for
15: return  $\text{dist}$  and  $\text{parent}$ 
```

Theorem 4.1. *After the k -th pass of edge relaxations, $\text{dist}[v] \leq \text{OPT}(k, v)$ for every v ; in particular, if the graph has no negative cycle, after $n - 1$ passes $\text{dist}[v]$ equals the true shortest-path distance from s to v . The algorithm runs in $O(mn)$ time.*

Proof. By induction on k . Initially $\text{dist}[s] = 0 = \text{OPT}(0, s)$ and all others are ∞ . Suppose the claim holds after pass $k - 1$. Consider any v and an optimal $\leq k$ -edge path P to v . If P uses $\leq k - 1$ edges, then by induction $\text{dist}[v]$ was already $\leq \text{OPT}(k - 1, v) = \text{cost}(P)$. Otherwise P has a last edge (u, v) and a prefix that is a $\leq k - 1$ edge path to u of cost $\text{OPT}(k - 1, u)$; by induction $\text{dist}[u] \leq \text{OPT}(k - 1, u)$ before pass k , and when pass k relaxes (u, v) it sets $\text{dist}[v] \leq \text{dist}[u] + w \leq \text{cost}(P)$. Since shortest paths in a negative-cycle-free graph are simple and use at most $n - 1$ edges, $\text{dist}[v]$ reaches the true distance after $n - 1$ passes. Each pass relaxes all m edges, giving $O(mn)$ total. $\square$

4.4 Negative-cycle detection

The correctness theorem says distances stabilise after $n - 1$ passes *when no negative cycle exists*. This gives a clean test (Problem 7): run one **extra** (n -th) pass; if *any* edge can still be relaxed (some $\text{dist}[u] + w < \text{dist}[v]$), then a shortest path of more than $n - 1$ edges would be improving on a simple path, which is only possible if a negative cycle is reachable. To recover the cycle itself, record the parent pointer that caused the successful relaxation in the extra pass, then follow parent pointers n times (to guarantee landing inside the cycle) and read off the cycle. Problem 6 implements the distance computation and Problem 7 the detection.

5 All-Pairs Shortest Paths: Floyd–Warshall

5.1 Problem and recurrence

Sometimes we want shortest paths between *every* pair of vertices. Running Bellman–Ford from each source costs $O(mn^2)$; for dense graphs the **Floyd–Warshall** algorithm is simpler and competitive at $O(n^3)$. Its DP is indexed by the set of *intermediate vertices* allowed on a path.

Number the vertices $1, \dots, n$. Let $d_k(i, j)$ be the length of the shortest path from i to j whose intermediate vertices all lie in $\{1, \dots, k\}$. Considering whether an optimal such path uses vertex k as an intermediate:

$$d_k(i, j) = \min(d_{k-1}(i, j), d_{k-1}(i, k) + d_{k-1}(k, j)),$$

with base case $d_0(i, j) = w(i, j)$ if $(i, j) \in E$, $d_0(i, i) = 0$, and ∞ otherwise. Either the optimal path avoids k (first term), or it goes $i \rightsquigarrow k \rightsquigarrow j$ through k exactly once, splitting into two paths each using only $\{1, \dots, k - 1\}$ as intermediates (second term). The final answer is $d_n(i, j)$.

Algorithm 3 Floyd–Warshall (all-pairs shortest paths)

```

1:  $d[i][j] \leftarrow w(i, j)$  for edges;  $d[i][i] \leftarrow 0$ ;  $\infty$  otherwise
2: for  $k = 1$  to  $n$  do
3:   for  $i = 1$  to  $n$  do
4:     for  $j = 1$  to  $n$  do
5:       if  $d[i][k] + d[k][j] < d[i][j]$  then
6:          $d[i][j] \leftarrow d[i][k] + d[k][j]$ 
7:       end if
8:     end for
9:   end for
10: end for
11: return  $d$ 
```

As with Bellman–Ford, a single matrix is updated in place; the correctness of doing so rests on the fact that the k -th outer iteration only *reads* entries that are still correct for the “ $\leq k$ intermediates” subproblem. A **negative cycle** reveals itself as a strictly negative diagonal entry $d[i][i] < 0$ after the algorithm finishes. Problem 8 implements Floyd–Warshall, and Problem 9 cross-checks it against Bellman–Ford: on any graph with no negative cycle, Bellman–Ford from source s must agree, vertex by vertex, with row s of the Floyd–Warshall matrix.

6 Shortest and Longest Paths in a DAG

6.1 Topological order makes DP trivial

In a **directed acyclic graph** there are no cycles at all – in particular no negative cycles – so even longest paths are well-defined (unlike in general graphs, where longest-simple-path is NP-hard). A topological order $v_1, v_2, \dots, v_n$ (every edge goes from earlier to later in the order; recall Week 3) lets us process vertices in an order where, by the time we reach v , all of v ’s predecessors are already finalised. The DP is then a single sweep:

$$\text{dist}(v) = \min_{(u,v) \in E} (\text{dist}(u) + w(u, v)), \quad \text{dist}(s) = 0,$$

processed in topological order. For *longest* paths, replace min with max (and initialise unreachable vertices to $-\infty$). Because each edge is examined once, the whole computation is $\mathbf{O}(\mathbf{n} + \mathbf{m})$ – faster than both Dijkstra ($O(m \log n)$) and Bellman–Ford ($O(mn)$), and it handles negative weights for free.

Algorithm 4 DAG Shortest Path (relax in topological order)

```

1: Compute a topological order of the vertices
2:  $\text{dist}[s] \leftarrow 0$ ;  $\text{dist}[v] \leftarrow \infty$  for all  $v \neq s$ 
3: for each vertex  $u$  in topological order do
4:   for each edge  $(u, v, w)$  leaving  $u$  do
5:     if  $\text{dist}[u] + w < \text{dist}[v]$  then
6:        $\text{dist}[v] \leftarrow \text{dist}[u] + w$ ;  $\text{parent}[v] \leftarrow u$ 
7:     end if
8:   end for
9: end for
10: return  $\text{dist}$  and  $\text{parent}$ 
```

This is the right tool when a problem has an inherent acyclic structure – task scheduling along dependencies, the critical path in a project (a *longest* path), or counting paths in a grid. Problem 10 implements DAG shortest paths and checks them against the more general Bellman–Ford, confirming that on a DAG the simpler linear-time sweep suffices where Dijkstra-style machinery would be overkill.

7 Two More Classic 2-D Dynamic Programs

7.1 Matrix chain multiplication

Given a chain of matrices $A_1 A_2 \dots A_n$ with A_i of dimension $p_{i-1} \times p_i$, matrix multiplication is associative, so any parenthesisation yields the same product – but the *cost* (number of scalar

multiplications) varies dramatically. The DP is indexed by sub-chains: let $M(i, j)$ be the minimum cost of multiplying $A_i \cdots A_j$. Reasoning about the *outermost* multiplication (the last split point k):

$$M(i, j) = \min_{i \leq k < j} (M(i, k) + M(k + 1, j) + p_{i-1} p_k p_j), \quad M(i, i) = 0.$$

There are $O(n^2)$ subproblems, each minimising over $O(n)$ split points, for $O(n^3)$ time. Recording the optimal split k for each (i, j) lets us reconstruct the optimal parenthesisation as a string (Problem 11). This is the textbook example of DP over *contiguous sub-intervals*, a third indexing pattern beyond prefixes and vertex sets.

7.2 Grid paths

Consider an $r \times c$ grid where you may move only *right* or *down*. Two classic questions: (a) how many distinct paths lead from the top-left to the bottom-right corner, and (b) if each cell has a cost, what is the *minimum-cost* path? Both are one-line recurrences. For counting,

$$P(i, j) = P(i - 1, j) + P(i, j - 1), \quad P(0, 0) = 1,$$

and for minimum path sum over cell costs $c(i, j)$,

$$S(i, j) = c(i, j) + \min (S(i - 1, j), S(i, j - 1)),$$

each filled in $O(rc)$ time. The grid is implicitly a DAG (every move increases $i + j$), so these are just the DAG recurrences specialised to a grid; an optimal path is recovered by the usual traceback (Problem 12).

8 Summary and Looking Ahead

This week extended dynamic programming from one-dimensional to two-dimensional (and interval- and vertex-set-indexed) subproblems, through several fundamental algorithms:

- **Sequence alignment / edit distance:** the $O(mn)$ two-dimensional recurrence over prefix pairs, with $O(m+n)$ traceback for the alignment, and Hirschberg's $O(m+n)$ -space refinement.
- **Longest common subsequence:** a sibling recurrence, related to insert/delete edit distance by $d = m + n - 2\text{LCS}$.
- **Bellman–Ford:** shortest paths with negative edges in $O(mn)$ via a DP on the number of edges used, plus a one-extra-pass test for negative cycles.
- **Floyd–Warshall:** all-pairs shortest paths in $O(n^3)$ via a DP on the set of allowed intermediate vertices.
- **DAG shortest/longest paths:** a single $O(n + m)$ sweep in topological order, handling negative weights and longest paths for free.
- **Matrix chain multiplication and grid paths:** DP over sub-intervals and over a grid, both with reconstruction.

The unifying lesson is the DP recipe: define subproblems, reason about the *last decision* to get a recurrence, order the computation so dependencies come first, and trace back to reconstruct an actual optimal solution.

Next week we turn to **network flow** (Kleinberg & Tardos, Ch. 7): the maximum-flow / minimum-cut problem and the Ford–Fulkerson method. Flow is a different algorithmic paradigm from DP, but it shares this week’s graph-theoretic setting and – as we will see – a great many seemingly unrelated problems (bipartite matching, image segmentation, project selection) reduce to it.